

Tri9T

Document Versioning, Ingestion, and Verification Engine

Prepared by: Tri9T Backend Engineering Team

Status: Production Verified

Document Version: 1.0.0

Target Platform: FastAPI + SQLite / MongoDB & Gemini LLM Pipeline

Date: July 2026

Table of Contents

1. Executive Summary & System Architecture	Page 3
2. Relational Database Schema	Page 3
3. PDF Parsing & Hierarchy Reconstruction	Page 4
4. Parser Edge Cases & Mitigations	Page 4
5. Document Versioning & Matching Strategy	Page 5
6. Stale Traceability & Content Hash Detection	Page 5
7. LLM QA Generation & Retry Strategy	Page 6
8. Technical Decision Log	Page 7
9. Known Limitations & Future Improvements	Page 7

System Overview

Tri9T is a production-grade automated verification platform designed to ingest complex technical PDF manuals, reconstruct their hierarchical outline structure, and manage logical node revisions over time. It enables QA test engineers to highlight specific sections of a manual, generate 3-5 high-fidelity test cases using Gemini LLM structured outputs, and automatically trace the staleness of these test cases as updated versions of the source manuals are uploaded.

1. Executive Summary & System Architecture

The platform employs a clean, decoupled service architecture to separate core ingestion, validation, and retrieval tasks:

- **API Router Layer:** Implements FastAPI routers for selections, document versions, node retrieval, and QA management. It strictly enforces Pydantic request and response models.
- **Service Layer:** Orchestrates parsing logic (via PyMuPDF and PyTesseract), tree reconstruction (via Stack-based hierarchy parsing), version comparisons, and LLM retry pipelines.
- **Persistence Layer:** Operates an asynchronous SQLite engine via aiosqlite for version control mappings, selections, and test case records. In parallel, MongoDB stores raw extracted page layouts and blocks.

2. Relational Database Schema

The database tracks the evolution of document outlines across multiple manual uploads. Rather than tracking changes line-by-line, the schema maps revisions to **Logical Nodes**. A logical node acts as a persistent anchor with a stable UUID. When a new document version is imported, the system maps matching outline sections back to the same logical node ID, creating a new NodeVersion record that links to the new DocumentVersion.

Table	Key Columns	Description
documents	id (PK), name	Represents the root document profile.
document_versions	id (PK), document_id (FK), version_number, commit_message	Captures a specific manual revision or upload transaction.
logical_nodes	id (PK), uuid, document_id (FK)	Stable anchor representing a structural node across all document versions.
node_versions	id (PK), logical_node_id (FK), document_version_id (FK), title, content, content_hash	Holds the actual text content, title, and SHA-256 hash of a node under a specific version.
selections	id (PK), document_version_id (FK), name	A user-selected subset of logical nodes pinned to a document version.
generated_test_cases	id (PK), selection_id (FK), question, answer, reference_context	Stores the validated, LLM-generated QA test cases linked to a selection.
llm_generation_failures	id (PK), selection_id (FK), raw_response, error_message	Logs failed LLM outputs and schema validation errors for audits.

3. PDF Parsing & Hierarchy Reconstruction

Parsing complex layout specifications (like the CT200 user manual) requires a five-step extraction pipeline:

Step 1: Raw Page Extraction & OCR Fallback

Pages are extracted using PyMuPDF. If a page yields no digital characters (indicating a scanned image), the system renders the page to a 150 DPI image and runs Tesseract OCR (pytesseract) to retrieve page text.

Step 2: Table Isolation

PyMuPDF's table extraction locates cells. The TableDetector resolves overlapping or nested bounding boxes by preserving only the largest enclosing table frame. Rows are formatted into clean Markdown grids.

Step 3: Geometry & Reading Order Alignment

LayoutAnalyzer merges text blocks and tables. It discards any text block falling inside table boundaries (to prevent duplicate table content extraction) and sorts all remaining elements top-to-bottom.

Step 4: Classification & Outline Stack Builder

BlockClassifier identifies headings (dot-separated keys e.g. '2.1.1.1' or numbers like '1.'), lists, and paragraphs. HierarchyBuilder merges paragraphs split across page boundaries and reconstructs the outline tree using an ancestor tracking stack. Mismatched headings emit warnings without silently assigning faulty parents.

4. Parser Edge Cases & Mitigations

Edge Case Scenario	Pipeline Mitigation Strategy
Missing intermediate headings E.g. heading 3.1.2 appears but 3.1 was skipped.	The HierarchyBuilder detects the missing parent key, raises a <i>Hierarchy mismatch</i> warning, and attaches the node to the longest matched prefix (e.g. heading 3.0 or root) to preserve order.
Page-boundary text split A paragraph breaks across page bounds.	Parser checks if the next block starts with a lowercase letter, a hyphen, or a list marker, automatically merging it into the previous paragraph block.
Duplicate headings Multiple headings share a key or name.	The system maintains local counters for each heading path and appends a duplicate suffix (<i>_dup[count]</i>) to enforce unique, stable logical signatures in the database.
Table text duplicate extraction Standard readers extract cell text twice.	The LayoutAnalyzer performs a spatial boundary check. If a text block's coordinate center falls within the bounding box of a detected table, it is dropped from the text stream.

5. Document Versioning & Matching Strategy

A key design requirement is recognizing logically identical nodes when a new document version is ingested. Rather than performing character-based text alignment, the system maps outline nodes based on structural path signatures.

- 1. Path Signature Construction:** During manual ingestion, a signature string is generated for each node representing its type, heading hierarchy, and order index relative to its parent (e.g., *heading:2.0 > heading:2.1 > paragraph:3*).
- 2. Map Alignment:** The signature is looked up in the previous version's signature-to-logical-node map (*prev_sig_map*).
- 3. Identity Mapping:** If found, the node is associated with the existing *logical_node_id*. This preserves history across re-uploads. If missing, a new *logical_node_id* is created.

Change Detection Rule Matrix

Scenario	Signature Path Match	Content Hash Match	Result Category
Identity Preserved, No Change	Matches Previous	Identical	Unchanged
Node Relocated in Manual	Different Path	Identical	Unchanged (Moved)
Section Modified in Place	Matches Previous	Differs	Modified
New Section Introduced	No Match (New)	N/A	Added
Section Removed / Deleted	No Match in V2	N/A	Removed

6. Stale Traceability & Content Hash Detection

A user's selection contains reference text pinned to specific logical nodes. When a new version of the manual is ingested, the system traces the selection validity by examining the state of the referenced nodes:

- **Fresh:** All selected logical nodes exist in the new version, and their content hashes (SHA-256) are identical to the source version.
- **Possibly Stale:** All selected logical nodes exist in the new version, but the content hash of one or more nodes has changed (text updated).
- **Stale:** One or more of the selected logical nodes are entirely absent in the new version (the section was deleted).

Note on Hash-based Limits: Content hashing is sensitive to character-level edits. Cosmetic corrections (such as changing 'Safety Guidelines' to 'Safety Guideline') trigger a 'Possibly Stale' flag, even though the underlying QA test cases remain semantically valid (False Positives).

7. LLM QA Generation & Retry Strategy

The LLM integration transforms selection snippets into functional QA test cases. To guarantee safety and structure, the generation pipeline is designed with strict boundaries:

Structured Output JSON Validation

The Gemini model is prompted to act as a Senior QA Automation Engineer. It is provided with the text context and instructed to return a structured JSON response matching the following JSON Schema:

```
{
  "properties": {
    "test_cases": {
      "items": {
        "properties": {
          "question": { "type": "string" },
          "answer": { "type": "string" },
          "reference_context": { "type": "string" }
        },
        "required": ["question", "answer", "reference_context"],
        "type": "object"
      },
      "type": "array"
    }
  },
  "required": ["test_cases"],
  "type": "object"
}
```

Retry and Audit Execution Flow

LLM outputs are cleaned of markdown formatting markers (like triple-backtick 'json') and parsed. If a JSON parsing failure or a Pydantic schema validation error occurs, the system triggers **exactly one retry**. If the retry also fails, the system logs the raw response and error string to the *llm_generation_failures* table and returns an HTTP 422 error, providing transparency and audit capability.

Example System Instruction Prompt: You are a Senior QA Test Engineer. Generate 3-5 QA test cases directly based on the selected text. Provide accurate factual answers derived ONLY from the provided text context. Your output MUST be a JSON object conforming to the QAGenerationResponse schema. Do not include any conversational filler or preambles outside the JSON block.

8. Technical Decision Log

Q1: How does the system handle structural changes between versions?

Answer: The system separates structural outline mapping from content payloads. Outline nodes are assigned stable UUIDs on their first ingestion. Subsequent uploads resolve nodes against their parent outline paths (signatures) rather than absolute coordinates or index lines. This isolates structural shifts (e.g. moving a section) from content edits, allowing precise version history tracking.

Q2: Why was SQLite and SQL Schema chosen?

Answer: The core requirements—document outlines, logical anchors, selection nodes, test cases, and audit trails—are highly relational. A normalized SQL schema (using SQLAlchemy and SQLite withaiosqlite) enforces strict integrity, provides clean join operations, and requires zero external daemon setup. This makes development, integration testing, and local setups lightweight and reproducible. MongoDB is integrated in parallel as a secondary store for raw document layouts.

Q3: How is LLM integration validation structured?

Answer: Raw LLM responses are parsed dynamically, stripped of markdown formatting blocks, and validated using Pydantic models. A double-attempt execution path retries failures immediately. Persistent validation failures are captured in the database (llm_generation_failures) rather than being silently dropped.

9. Known Limitations & Future Improvements

Known Limitations:

- **Over-Sensitivity (False Positives):** Minor typo or spacing changes alter the content hash, marking selections as 'Possibly Stale' even if the test cases remain valid.
- **Out-of-Selection Context Blindness:** If a sibling warning changes but the selected node remains identical, the selection status remains 'Fresh' despite potential semantic conflicts.

Future Improvements:

- **Semantic Similarity Scoring:** Calculate cosine similarity between text embeddings of original and new sections. Flag changes below 0.98 similarity as modified, ignoring minor typo fixes.
- **AST & Relative Outline Hashing:** Hash the node's relative parent signature path along with the text. If a section is reordered, the selection status shifts to 'Possibly Stale' automatically.
- **Auto-Repair Pipeline:** Add an LLM worker that parses 'Possibly Stale' selections and auto-corrects QA test cases to match the newly ingested version.